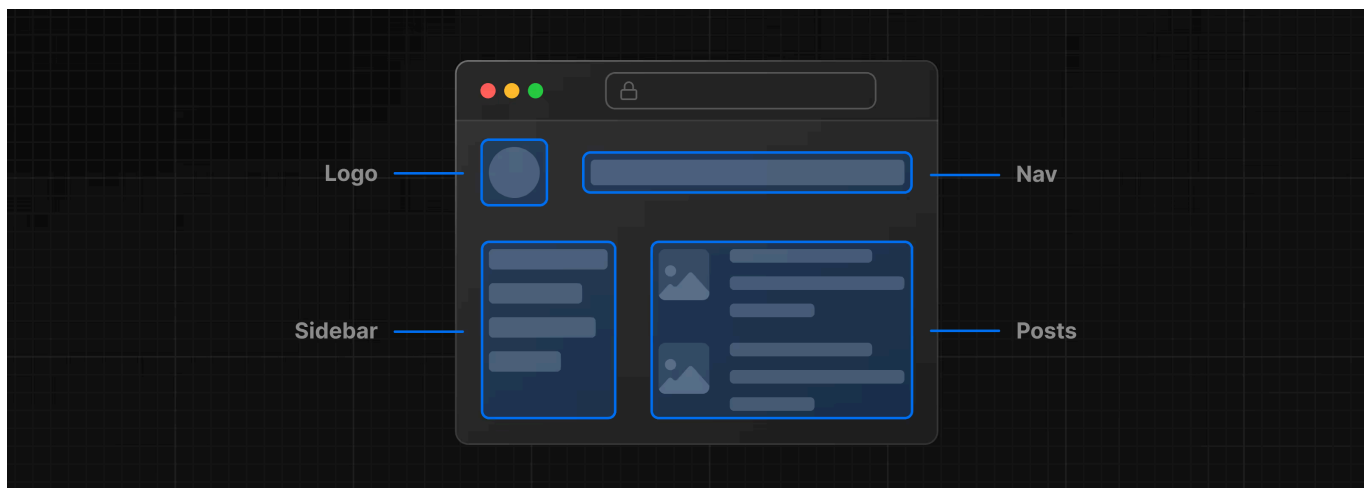


What is it

Building blocks of a web application

- **User Interface** - how users will **consume and interact** with your application.
- **Routing** - how users **navigate** between different parts of your application.
- **Data Fetching** - where your **data lives and how to get it**.
- **Rendering** - when and **where you render static or dynamic content**.
- **Integrations** - what third-party services you use (for CMS, auth, payments, etc.) and how you connect to them.
- **Infrastructure** - where you deploy, store, and run your application code (serverless, CDN, edge, etc.).
- **Performance** - how to optimise your application for end-users.
- **Scalability** - how your application adapts as your team, data, and traffic grow.
- **Developer Experience** - your team's experience building and maintaining your application.

React =



By library, we mean React provides helpful functions (APIs) to build UI, but leaves it up to the

developer where to use those functions in their application.

Why devs like React - because it doesn't control; use it or what you can add to it, it gives you the piece you need, customise d, and you can customise the rest.

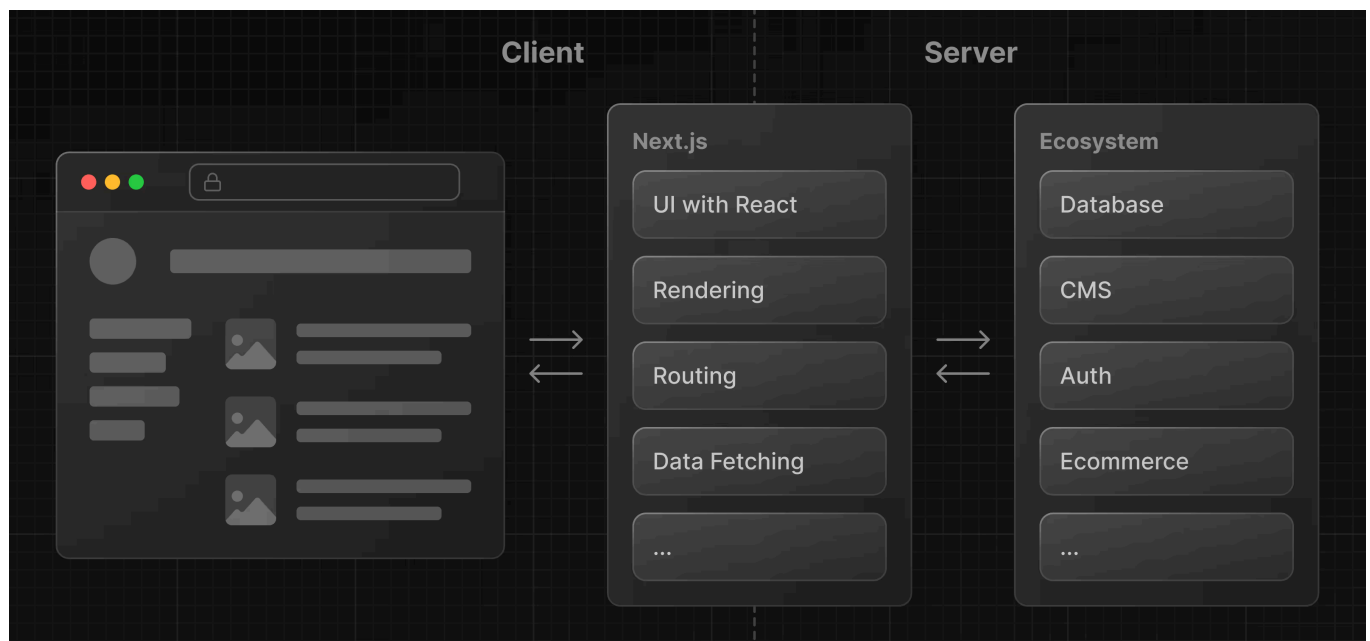
Why this can be bad

Starting new projects takes time because of the long configuration, wasting time solving **common problems and reinventing solutions. personal experience**

- - setting a bundler
- - ts file
- - routing
- - auth
- - using new tools e.g SSR

What is Next.js?

Next.js is a React **framework** that gives you building blocks to create web applications.



By framework, we mean **Next.js handles the tooling and configuration** needed for React, and **provides additional structure, features, and optimisations for your application.**

So we can:

1. use React for the UI
2. incrementally adopt Next.js features to solve common application requirements, e.g routing, data fetching, and caching
3. improve the developer and end-user experience.

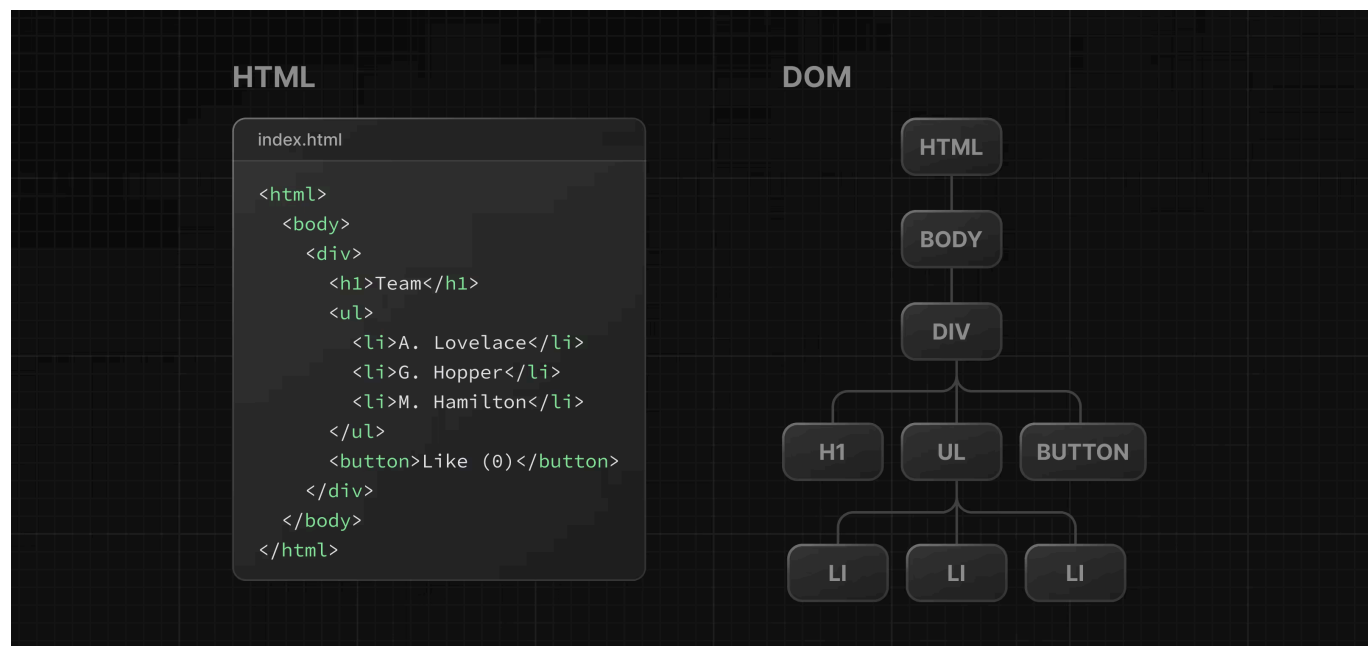
You can use React to build your UI, then incrementally adopt Next.js features to solve common application requirements such as routing, data fetching, and caching - all while improving the developer and end-user experience.

You can use React and Next.js to build **fully interactive, highly dynamic, and performant web applications**.

---New Chapter---

Rendering User Interfaces (UI)

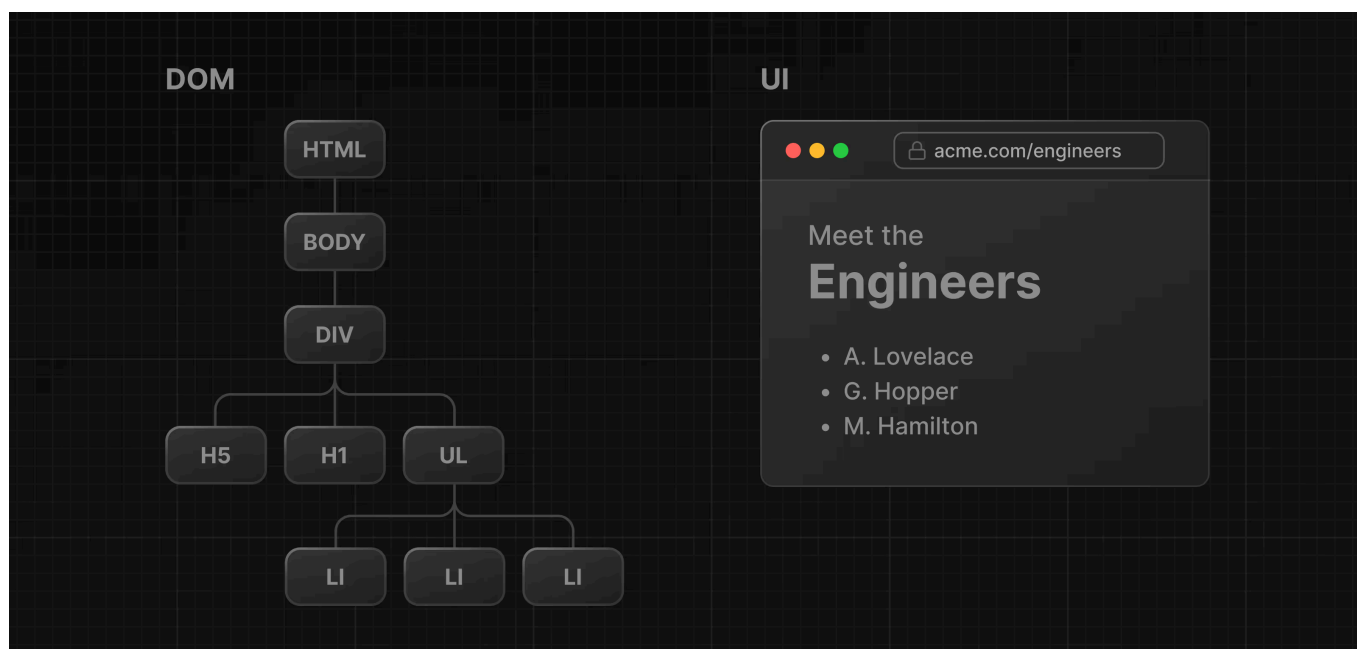
When a user visits a web page, the **server returns an HTML file** to the browser that may look like this:



The browser then reads the HTML and constructs the Document Object Model (DOM).

What is the DOM?

The DOM is an **object representation of the HTML elements**. It acts as a **bridge between your code and the user interface**, and has a tree-like structure with parent and child relationships.



You can use DOM methods and JavaScript, to **listen to user events** and **manipulate the DOM** by **selecting, adding, updating, and deleting specific elements in the user interface**. DOM manipulation allows you to not only target specific elements, but also change their style and content.

DOM vs HTML

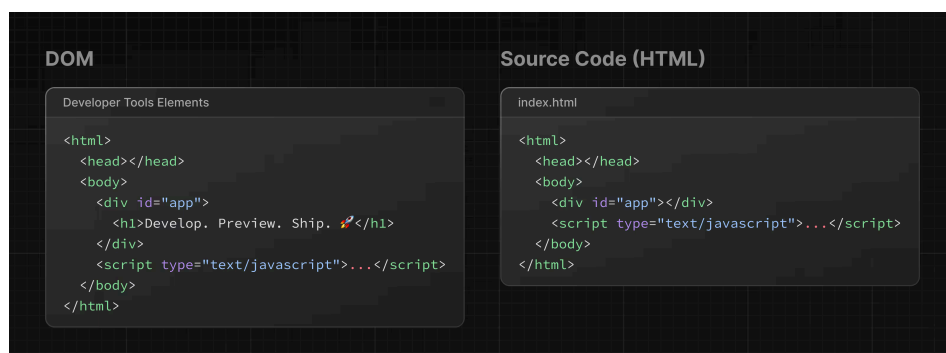
HTML - represent the initial page content

DOM - represents the current page content,

When js changes the dom, the dom becomes different from the html

HTML vs. the DOM

If you look at the DOM elements inside your **browser developer tools**, you will notice the DOM includes the `<h1>` element. **The DOM of the page is different from the source code - or in other words, the original HTML file you created.**



This is because the HTML represents the **initial page content**, whereas the DOM represents the **updated page content** which was **changed by the JavaScript code** you wrote. Updating the DOM with plain JavaScript is very powerful but **verbose**. You've written all this code to add

an `<h1>` element with some text:

```
index.html

<script type="text/javascript">
  const app = document.getElementById('app');
  const header = document.createElement('h1');
  const text = 'Develop. Preview. Ship.';
  const headerContent = document.createTextNode(text);
  header.appendChild(headerContent);
  app.appendChild(header);
</script>
```

As the size of an app or team grows, it can become increasingly chal

Problem

Updating the DOM with plain JavaScript is very powerful but verbose. You've written all this code to add an `<h1>` element with some text:

using more words than needed to express an idea.

- As the size of an app or team grows, it can become increasingly challenging to build applications this way.

With this approach, developers spend a lot of time writing instructions to tell the computer **how** it should do things. But wouldn't it be nice to describe **what** you want to show and let the computer figure out **how** to update the DOM?

Imperative vs. declarative programming

imperative programming - writing DOM methods to create a **reaction** to the DOM

declarative programming - Using pre-built DOM methods to interact with the DOM

Imperative — you write the step-by-step *how*. Grab the node, create an element, set properties, append it, and manually update it again when something changes.

js

```
const btn = document.createElement('button');
btn.textContent = `Count: ${count}`;
btn.addEventListener('click', () => {
  count++;
  btn.textContent = `Count: ${count}`; // you own the update
```

```
});  
document.body.appendChild(btn);
```

Declarative — you describe *what* the UI should be for a given state, and something else works out the DOM calls.

jsx

```
<button onClick={() => setCount(count + 1)}>Count: {count}</button>
```

The correction: declarative isn't "using pre-built DOM methods." `appendChild` and `classList.add` are pre-built and still imperative — you're still issuing the instructions yourself. What makes React declarative is that you never say "now update that text node"; you change state and React diffs and patches the DOM for you.

Under the hood it's all the same DOM API. The line is *who writes the mutation steps* — you, or the library.

Getting Started with React(Chapter - 4)

To use React in your project, you need two packages:

- `react` is the core React library.
- `react-dom` provides DOM-specific methods that enable you to use React with the DOM.

What is JSX?

JSX is a syntax extension for JavaScript that allows you to describe your UI in a familiar *HTML-like* syntax. The nice thing about JSX is that apart from following [three JSX rules](#), you don't need to learn any new symbols or syntax outside of HTML and JavaScript.

But browsers don't understand JSX out of the box, so you'll need a JavaScript compiler, such as Babel, to transform your JSX code into regular JavaScript.

You can start to see how using React enables you to cut down a lot of repetitive code.

And **it's a library that contains reusable snippets of code that perform tasks on your behalf** exactly what React does

React uses JSX, which needs to be compiled into JavaScript.

JSX is a syntax extension for JavaScript, but browsers don't understand JSX out of the box, so you'll need a JavaScript compiler to transform your JSX code into regular JavaScript.

Building UI with Components

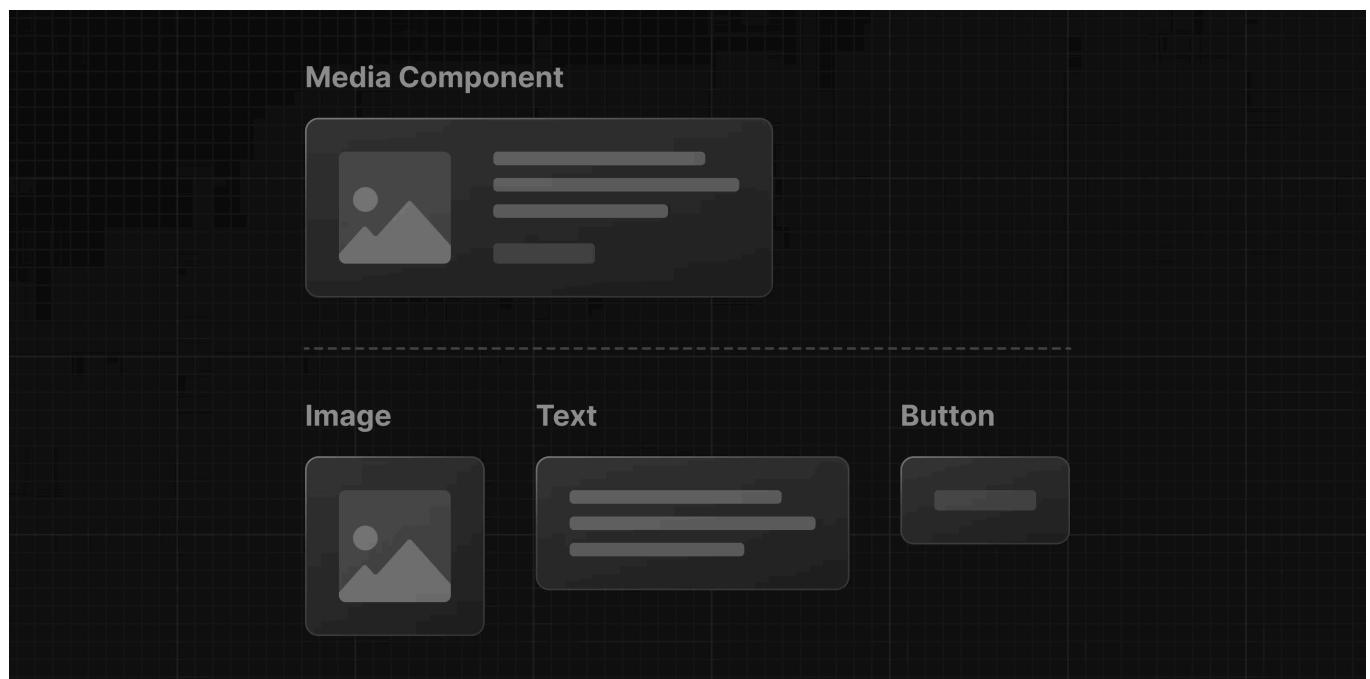
There are three core concepts of React that you'll need to be familiar with to start building React applications. These are:

- Components
- Props
- State
- Future: Server and Client Components

Components

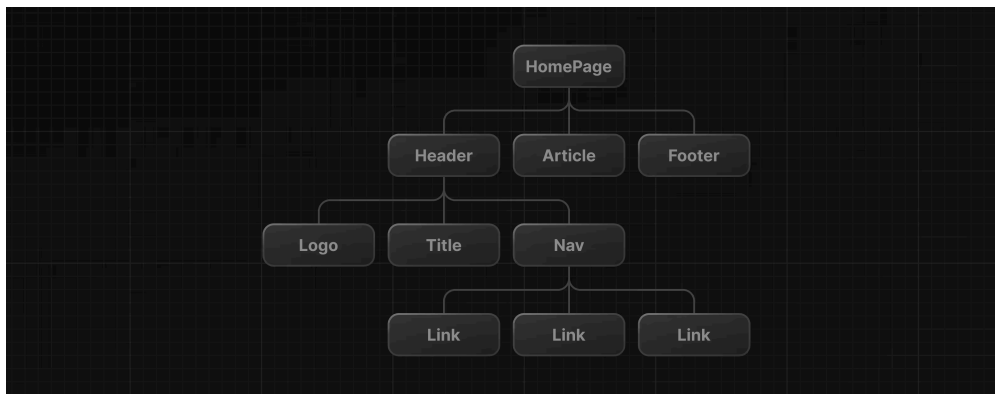
Small building blocks used to build UI.

Components allow you to build self-contained, **reusable snippets of code**. If you think of components as **LEGO bricks**, you can take these individual bricks and **combine them together to form larger structures**. If you need to **update a piece of the UI**, you can **update the specific component or brick**.



Component trees

You can keep nesting React components this way to form component trees.



For example, **your top-level** `HomePage` component could hold a `Header`, an `Article`, and a `Footer` Component. And each of those components could in turn have their own child components and so on. For example, the `Header` component could contain a `Logo`, `Title` and `Navigation` component.

This **modular format allows you to reuse components in different places** inside your app.

In React, you can nest components inside each other, forming a component tree.